Disruption: Finding and scraping news articles by search terms

The author of this repository is Clara Vandeweerdt. Contact info: clara.vandeweerdt@ifs.ku.dk

To run a module, run `main.py` in its main directory.

Module 1: Google URL scraping

Directory: google_scraping

This module takes one or more newspaper websites, one or more search strings, and a date range. It will search Google using the search strings using month-by-month date ranges, scrape the URLs from the results, combine and de-duplicate them, and write them to month-by-month csv files for each newspaper.

The script opens a non-headless Selenium Chrome webdriver, because it is necessary for a human to solve the captchas that Google throws.

csv files are stored in the article_urls folder, in a subfolder named after the newspaper.

Installting the Chrome WebDriver

The module uses Selenium with the Chrome WebDriver, so you need to download the ChromeDriver before use. You can download it from the following link: Chrome WebDriver. For newer versions of Chrome, it can be downloaded here: Chrome for testing (scroll down). Instructions on getting started (especially Python finding the executable) are available here.

Module 2: Content scraping

Directory: content_scraping

This module takes one or more newspapers and does three things: (1) scrape the contents of the urls obtained from Google for that newspaper, (2) parse the contents based on newspaper-specific parsing scripts, and (3) download the images (article photos) found in that parsed content. Running its `main.py` script will do all three, with options to re-do (or not) the steps for articles that have already been scraped, parsed and image-downloaded.

Disclaimer: newspapers may at any point change their approach to denying requests from bots and the layout of their article pages. This can make the scraping and the parsing scripts for that

newspaper less functional.

(1) Scraping

The script `scraping_articles/scraping_general_functions.py` contains functions for scraping article contents. Its main function is `main_scrape_html()`, which reads in a file with urls scraped from Google, visits them, scrapes their full html content, and stores those in a dictionary (which it outputs and writes to a json file). It decides what kind of scraper session to use for this depending on the news outlet: either `requests` or `selenium` (needed for bot detection or to interact with elements on the page, e.g. cookie banners).

The key function that `main_scrape_html()` calls is `fetch_url()`, which uses the session to retrieve the page html.

How to Use the Scraper for Outlets That Require a Subscription

To do this, we need to start a browser session, log in to the outlet's website (using a paid subscription), and save cookies. However, this only works if we will be using `requests` to scrape the article contents, not when using `selenium.`

First, in the `saved_sessions` folder, open the script called `Save_session.py`. Update the script to apply `save_session()` to the right newspaper.

Launch this script. It will open a Chrome browser. Navigate to the website of the newspaper and log in to your account.

Ensure that you accept cookies to be saved. Once you are logged in, you can close the Chrome window.

The cookies from this session will be stored and used to access the URLs listed in the list of URLs. This allows you to access all the subscriber's content.

You can now proceed to use the scraping code. If you find that the scraped content is inaccessible because you are not seemingly logged in, follow the steps above again.

(2) Parsing

The script `scraping_articles/parsing_general_functions.py` contains functions for parsing article contents. Its main function is `main_parse_content()`. It will call a specialized parsing module, depending on the newspaper. It will then use the `extract()` function from that module to take an article's html and tries to retrieve the title, subtitle, body text, images (captions and urls), author, and date.

It produces two dictionaries, which it also writes to json files: one for the items that were parsed successfully, and one for items that were parsed but dropped (e.g. because there was no body text or search terms were not found in the text). The dictionaries have one entry per article, with the key being an article ID created from the outlet name, date, and first words of the article title.

The newspaper-specific parsing modules can be found in the `scraping_articles/parsing_specific` directory.

(3) Image downloading

For the articles that were parsed successfully and not dropped, we download all the images. The key function for this is `main_download_pics()`, located in the `scraping_articles/scraping_general_functions.py` file. It will take the url that was parsed as the `src` of each image, download its contents, and store them in an `article_images` folder. It wil also update the parsed article dictionaries (and json files), adding a local name and path for each image in each article.

Other elements

- `scraping_articles/test.py`: tests the whole scraping--parsing--image downloading pipeline for one url at one newspaper
- `data_structuring/data_structuring_functions.py`: contains helper functions that read files, write objects to files (e.g. dict to json), and download files
- `article_presence_check.py`: after scraping, checks whether the urls retrieved from Google ended up as successfully parsed urls, dropped urls, or neither
- `logging_config.py`: sets up a logger which, when called, makes sure all logs (of level INFO and up) are written to a file named `scraping.log`